IL PROBLEMA DELL'ORDINAMENTO

SIA $A = \langle a_1, a_2, \dots, a_m \rangle$ UNA SEQUENZA DI m NUMERI INTERI ($a_i \in \mathbb{Z} \; \forall \, 1 \le i \le m$). ORDINARE A SIGNIFICA DETERMINARE UNA SEQUENZA $A' = \langle a'_1, a'_2, \dots, a'_m \rangle$ DETTA PERMUTAZIONE ORDINATA DI A, TALE CHE: $a'_1 \le a'_2 \le \dots \le a'_m$

INEFFICIENZA DELL'APPROCCIO BRUTEFORCE E LIMITI DI RIFERIMENTO

- ESISTENZA DELLA SOLUZIONE: POICHÉ LA RELAZIONE D'ORDINE SUI NUMERI È TOTALE, ESISTE SEMPRE UNA PERMUTAZIONE ORDINATA.
- COSTO BRUTE FORCE: UN ALGORITMO BRUTEFORCE CHE GENERI TUTTE LE POSSIBILI PERMUTAZIONI E VERIFICHI PER CIASCUNA LA CONDIZIONE DI ORDINAMENTO SCORRENDO I SUOI m ELEMENTI RICHIEDE UN TEMPO PARI A $\Omega(m \cdot m!)$.
- INTERVALLO DEGLI ALGORITMI BASATI SU CONFRONTI: OPERANDO TRAMITE IL CONFRONTO ITERATIVO DI COPPIE DI ELEMENTI, IL LIMITE SUPERIORE DEGLI ALGORITMI "CHE HA SENSO UTILIZZARE" È $T(m) = O(m^2)$, QUELLO INFERIORE INVECE - NEL CASO MIGLIORE ASSOLUTO - È $\Omega(m)$

INSERTION SORT

L'ALGORITMO INSERTION SORT ADOTTA UN APPROCCIO INCREMENTALE:

- AD OGNI ITERAZIONE DELL'INDICE ESTERNO j (CHE SCORRE DA 2 A m), LA SEQUENZA È DIVISA LOGICAMENTE IN DUE PARTI: IL SOTTOARRAY SINISTRO $A[1, \dots, j-1]$ GIÀ ORDINATO E IL SOTTOARRAY DESTRO $A[j, \dots, m]$ ANCORA DA ORDINARE
- L'ALGORITMO ESTRAE L'ELEMENTO CORRENTE $elem = A[j]$ E LO INSERISCE NELLA POSIZIONE CORRETTA ALL'INTERNO DELLA SOTTOSEQUENZA ORDINATA $A[1, \dots, j-1]$, SHIFTANDO A DESTRA TUTTI GLI ELEMENTI MAGGIORI DI $elem$
- ORDINAMENTO IN-PLACE: NON SONO NECESSARIE STRUTTURE DATI AGGIUNTIVE. L'ALGORITMO OPERA SU A, USANDO UN NUMERO COSTANTE DI VARIABILI SCALARI AUSILIARIE

```
INSERTIONSORT(A, n)
    FOR j = 2 TO n DO
        ELEM = A[j]
        //INSERIMENTO DI A[j] NELLA SEQUENZA ORDINATA
        i = j - 1
        WHILE i ≥ 1 && ELEM < A[i] DO
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = elem
```

ANALISI DELLA COMPLESSITÀ TEMPORALE

IL TEMPO DI ESECUZIONE DI INSERTION SORT NON DIPENDE SOLO DALLA DIMENSIONE DI n, MA ANCHE DALLA QUALITÀ DI ORDINAMENTO INIZIALE, POICHÉ NON È NOTO A PRIORI QUANTE VOLTE VERRÀ ESEGUITO IL CICLO WHILE.

PER LA FORMALIZZAZIONE DEL CALCOLO SI INTRODUCE IL PARAMETRO t_j, CON $j \in \{2, \dots, n\}$, DEFINITO COME IL NUMERO DI VALUTAZIONI DELLA CONDIZIONE DI TESTA DEL CICLO WHILE FISSATO L'INDICE j:

- LA TESTA DEL WHILE VIENE ESEGUITA $\sum_{j=2}^{n} t_j$ VOLTE
- IL CORPO VIENE ESEGUITO UNA VOLTA IN MENO RISPETTO AL TEST DI GUARD, OVVERO $\sum_{j=2}^{n} (t_j - 1)$ VOLTE

SOMMANDO IL COSTO DELLE ISTRUZIONI ELEMENTARI MOLTIPLICATO PER IL NUMERO DI ESECUZIONI, SI OTTIENE L'EQUAZIONE GENERALE PARAMETRICA

$$T_n = 9n - 7 + 4 \sum_{j=2}^{n} t_j + 4 \sum_{j=2}^{n} (t_j - 1)$$

DA CUI DERIVANO I 3 CASI FONDAMENTALI:

1. CASO MIGLIORE - ARRAY GIÀ ORDINATO ($\Theta(n)$)

   LA CONDIZIONE $ELEM < A[i]$ RISULTA IMMEDIATAMENTE FALSA, PER OGNI j, AL PRIMO CONTROLLO. SI HA QUINDI $t_j = 1$ PER OGNI $j = 2, \dots, n$. LE SOMMATORIE SI RIDUCONO A TERMINI LINEARI, RESTITUENDO $T(n) = \Theta(n)$

2. CASO PEGGIORE - ARRAY ORDINATO IN SENSO INVERSO ($\Theta(n^2)$)

IN QUESTO CASO LA CONDIZIONE ELEM < A[j] È SEMPRE VERA: OGNI ELEMENTO A[j] DEVE ESSERE CONFRONTATO CON TUTTI I $j-1$ ELEMENTI DI $A[1,\dots,j-1]$ FINO A FAR SCENDERE i A 0. SFRUTTANDO LA SERIE ARITMETICA DI GAUSS $\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1$ E $\sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$ SI OTTIENE

$$T(n) = 9n - 7 + 4\left(\frac{n(n+1)}{2} - 1\right) + 4\,\frac{n(n-1)}{2} = 6n^2 + 11n - 15 = \Theta(n^2)$$

3. CASO MEDIO - INPUT EQUIPROBABILI ($\Theta(n^2)$)

ASSUMENDO L'EQUIPROBABILITÀ DELLE ISTANZE, IL NUMERO DI CONFRONTI PER INSERIRE A[j] NELLA SOTTOSEQUENZA $A[1,\dots,j-1]$ È LA MEDIA ARITMETICA DEI VALORI DI t_j DA 1 A j

$$t_{medio} = \frac{\sum_{k=1}^{j} k}{j} = \frac{\frac{j(j+1)}{2}}{j} = \left\lfloor \frac{j+1}{2} \right\rfloor$$

SOSTITUENDO t_{medio} NELL'EQUAZIONE PARAMETRICA, LE SOMMATORIE RIMANGONO FUNZIONI LINEARI DI j, PRODUCENDO COMUNQUE UNA SOMMA QUADRATICA $T_{medio}(n) = \Theta(n^2)$

SELECTION SORT

L'ALGORITMO SELECTION SORT RAPPRESENTA IL DUALE LOGICO DI INSERTION SORT:

- FISSA A PRIORI LA POSIZIONE DA RIEMPIRE, CERCANDO NEL RESTO DELL'ARRAY L'ELEMENTO DA COLLOCARVI
- PARTENDO DALL'ULTIMA CELLA DELL'ARRAY, SCENDENDO FINO A $i = 2$, L'ALGORITMO INVOCA FINDMAX(A, i) PER INDIVIDUARE L'INDICE j DELL'ELEMENTO MASSIMO NEL SOTTOARRAY NON ORDINATO, PER POI SCAMBIARLO CON L'ELEMENTO IN POSIZIONE i
- IL CICLO SI ARRESTA A $i = 2$ POICHÉ, UNA VOLTA POSIZIONATI $n-1$ ELEMENTI PIÙ GRANDI NELLE CELLE $2,\dots,n$, L'ELEMENTO MINIMO SI TROVA GIÀ AUTOMATICAMENTE NELLA CELLA 1

```
SELECTION SORT (A, m)
    FOR i = m DOWN TO 2 DO
        j = FINDMAX (A, i)
        SWAP (A, i, j)
```

```
FINDMAX (A, m)
    MAX = 1
    FOR i = 2 TO m DO
        IF A[MAX] < A[i]
            max = i
    RETURN MAX
```

ANALISI DELLA COMPLESSITÀ

COSTO COMPUTAZIONALE ($\Theta(m^2)$ IN TUTTI I CASI): LA PROCEDURA FINDMAX SCORRE SEMPRE L'INTERO SOTTOARRAY DA 2 A i, EFFETTUANDO ESATTAMENTE $i-1$ CONFRONTI, CON COSTO $T_{FM}(i) = \Theta(i)$. IL COSTO COMPLESSIVO DI SELECTION SORT È QUINDI

$$T_C(m) = \Theta(m) + \sum_{i=2}^{m} T_{FM}(i) + \Theta(1) = \Theta m + \Theta\left(\left(\sum_{i=1}^{m} i\right) - 1\right) = \Theta\left(\frac{m(m-1)}{2} - 1\right) = \Theta(m^2)$$